Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Funciones, modularización y manejo de formularios en PHP

v 1.0

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Tabla de contenido

Tabla de contenido *1*

Funciones en PHP ***2***

Introducción **2**

Sintaxis básica **2**

Argumentos **2**

Argumentos pasados referencia 3

Valores de argumentos predeterminados 3

Devolver valores **3**

Modularización del código ***4***

Funciones include y require **4**

include 4

require 4

Versiones _once 4

Manejo de formularios ***6***

Introducción **6**

Métodos de envío de variables **6**

Variables superglobales $_POST y $_GET **6**

Subida de archivos **7**

Funciones en PHP

Introducción

Una función es un bloque de código con un nombre, que puede recibir datos (parámetros), realizar operaciones y devolver un resultado (o no).
Su objetivo principal es evitar la repetición de código y facilitar la lectura y el mantenimiento gracias a:

- Reutilización: escribir una vez, usar muchas veces.
- Organización: separa tareas en bloques lógicos.
- Mantenimiento: si hay que hacer un cambio, se modifica en un solo lugar.
- Claridad: el código se entiende mejor.

Sintaxis básica

Una función puede ser definida usando una sintaxis como la siguiente:

```php
<?php

    function foo($arg_1, $arg_2, ..., $arg_n) {
        echo "Función de ejemplo.\n";
        return $valordevuelto;
    }

?>
```

Cualquier código PHP válido puede aparecer dentro de una función, incluso otras funciones y definiciones de clases.
Los nombres de las funciones siguen las mismas reglas que otras etiquetas de PHP. Un nombre de función válido comienza con una letra o guion bajo, seguido de cualquier número de letras, números, o guiones bajos.

Argumentos

La información puede ser pasada a las funciones mediante la lista de argumentos, la cual es una lista de expresiones delimitadas por comas.
PHP soporta argumentos pasados por valor (por defecto), pasados por referencia, y valores de argumentos predeterminados.
Ejemplo de matrices como argumentos de una función

```php
<?php

    function tomar_matriz($entrada) {
        echo "$entrada[0] + $entrada[1] = ", $entrada[0]+$entrada[1];
    }

?>
```

Argumentos pasados referencia

Por defecto, los argumentos de las funciones son pasados por valor (por lo que, si el valor del argumento dentro de la función se cambia, no se cambia fuera de la función). Para permitir a una función modificar sus argumentos, éstos deben pasarse por referencia.
Para hacer que un argumento a una función sea siempre pasado por referencia hay que poner delante del nombre del argumento el signo 'ampersand' (&) en la definición de la función:
Ejemplo de argumento pasado por referencia

```php
<?php

    function añadir_algo(&$cadena) {
        $cadena .= 'y algo más.';
    }

    $cad = 'Esto es una cadena, ';
    añadir_algo($cad);
    echo $cad;    // imprime 'Esto es una cadena, y algo más.'
?>
```

Valores de argumentos predeterminados

Una función puede definir valores predeterminados al estilo C++ para argumentos escalares como sigue:
Ejemplo de parámetros predeterminados en funciones

```php
<?php

    function hacercafé($tipo = "capuchino") {
        return "Hacer una taza de $tipo.\n";
    }

    echo hacercafé();
    echo hacercafé(null);
    echo hacercafé("espresso");
?>
```

Devolver valores

Los valores son devueltos usando la sentencia opcional return. Se puede devolver cualquier tipo, incluidos arrays y objetos. Esto causa que la función finalice su ejecución inmediatamente y pase el control de nuevo a la línea desde la que fue llamada.
Si se omite return() será devuelto el valor NULL.
Una función no puede devolver múltiples valores, pero se pueden obtener resultados similares

Modularización del código

Modularizar significa dividir un programa en partes más pequeñas y manejables, llamadas módulos.
En PHP, esto suele lograrse guardando diferentes partes del código en archivos separados y luego incluyéndolos en el archivo principal.
Ventajas de modularizar

- Facilita el trabajo en equipo: cada integrante puede trabajar en un módulo diferente.
- Reutilización de código: los módulos pueden usarse en distintos proyectos.
- Mantenimiento simplificado: se localizan y corrigen errores más rápido.
- Claridad: el código principal se mantiene limpio y fácil de leer.

Estos módulos pueden ser bloques de código, bibliotecas de funciones, librerías, etc.

Funciones include y require

En PHP, para incorporar módulos usamos estas funciones que en líneas generales realizan la inclusión de un archivo PHP al flujo actual de ejecución pero difieren levemente en algunas sutilezas que ya mencionaremos

include

Incluye y evalúa el archivo especificado.
Si el archivo no existe, lanza una advertencia (warning) pero el script continúa ejecutándose.

```php
<?php

    include 'librerias.php';


?>
```

require

También incluye y evalúa un archivo, pero si el archivo no existe o hay un error, produce un error fatal y detiene la ejecución

```php
<?php

    require 'librerias.php';


?>
```

Versiones _once

include_once y require_once hacen lo mismo, pero verifican que el archivo no se incluya más de una vez, evitando problemas como redefinición de funciones o variables.

A continuaciones se deja un ejemplo de cómo se implementaría la modularización en un bloque de código PHP

```php
<?php

    // index.php
    require 'header.php';
    include 'funciones.php';

    echo "<h1>Bienvenidos a DAW</h1>";

    require_once 'footer.php';

?>
```

Manejo de formularios

Introducción

Los formularios HTML son un mecanismo muy útil para que el usuario frontEnd interactúe con el servidor, y por ende, con nuestra aplicación. Constantemente nosotros como usuarios estamos llenando y enviando formularios y ya lo hemos naturalizado pero ¿Que sucede cuando se envía un formulario? Eso es lo que se va a explicar en esta unidad y se aprovechará a introducir varios temas relacionados con PHP del lado del servidor.

Métodos de envío de variables

Existen dos métodos HTTP que un cliente puede utilizar para pasar los datos del formulario al servidor: GET y POST. El método que utiliza un formulario particular, se especifica con el atributo *method* en la etiqueta *form*.
Una solicitud GET codifica los parámetros del formulario en la dirección URL mientras que una solicitud POST asa los parámetros del formulario en el cuerpo de la solicitud HTTP, dejando intacta la URL
Mas allá del método del envío, tenemos otra configuración que es el método de codificación de la información enviada del formulario. El HTML proporciona tres métodos de codificación.

- application/x-www-form-urlencoded (el valor predeterminado)
- multipart/form-data
- text/plain

Mas Adelante mencionaremos un poco de esto.

Variables superglobales $_POST y $_GET

$_GET y **$_POST** son variables superglobales que forman arrays de keys y values, donde los keys son los nombres del formulario (atributo "name") y los values son los datos de entrada de los usuarios. El hecho de que sean **superglobals** hace que sean accesibles desde el script independientemente del ámbito.
$_GET es un array de variables que se pasan al script a través de los **parámetros de URL**. La información que se envía es visible para todo el mundo y tiene limitada la cantidad de información que se puede enviar a 2000 caracteres. Las URLs con los datos enviados pueden guardarse en marcadores, lo que puede ser útil en ciertos casos. $_GET sólo se emplea para el envío de información reducida y no sensible.
$_POST es un array de variables que se pasan al script a través del método **HTTP POST**. La información que se envía no es visible para los demás ya que los nombres y variables van

embebidas en el **body** del **HTTP request**. No tiene límites en la cantidad de información a enviar. Soporta datos de entrada **multi-part binary** a la hora se subir archivos al servidor

Supongamos el siguiente formulario:

```
<html>
<body>
<form action="formpost.php" method="post">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
</body>
</html>
```

La lectura de los datos enviados por el formulario desde el script formpost.php podría ser algo parecido a esto:

```
<html>
<body>
Hola <?php echo $_POST["nombre"]; ?><br>
Tu email es: <?php echo $_POST["email"]; ?>
</body>
</html>
```

Si hiciéramos un pequeño cambio en el método de envió del formulario para enviar ahora la información por el método GET la lectura de la información enviada sería en cambio:

```
<html>
<body>
Hola <?php echo $_GET["nombre"]; ?><br>
Tu email es: <?php echo $_GET["email"]; ?>
</body>
</html>
```

Subida de archivos

Vamos a avanzar un poco más en los conceptos para introducir al manejo de archivos. Para poder subir archivos desde un formulario analicemos primero el siguiente ejemplo:

```
1   <html>
2   <body>
3   <form action="upload.php" method="POST" enctype="multipart/form-data">
4       <input type="file" name="imagen" />
5       <input type="submit" name="submit" />
6   </form>
7   </body>
8   </html>
9
```

Como se observa, además del envío de información por el método POST, el formulario tiene explícitamente el atributo `enctype="multipart/form-data"`. Es gracias a este atributo que habilitaremos el envío de archivos al servidor.
La lectura de los archivos enviados no difiere demasiado de la forma en que se lee la información de cualquier otra variable del formulario. En PHP nos encontraremos con la variable superglobal **$_FILES** que no es más que un array asociativo similar a los arreglos $_POST y $_GET. Este array se crea con el **key** que se indique en el input del formulario, en este caso "imagen":

- $_FILES["imagen"]**["name"]**. Guarda el nombre original del archivo del cliente.
- $_FILES["imagen"]**["type"]**. Guarda el MIME type del archivo.
- $_FILES["imagen"]**["size"]**. Guarda el tamaño del archivo en bytes.
- $_FILES["imagen"]**["tmp_name"]**. Guarda el nombre del archivo temporal.
- $_FILES["imagen"]**["error"]**. Guarda cualquier código de error que pueda provocar la subida del archivo.

Toda la información del archivo subido puede ser fácilmente accesible desde el script PHP. Cabe destacar que todo archivo enviado mediante un formulario se almacena en un directorio temporal en el servidor. Será necesario mover ese archivo al path de la aplicación para poder tener en forma permanente el file
La función move_uploaded_file() mueve un archivo subido del directorio temporal al directorio que se indique.

```php
<?php

if(isset($_POST["submit"]) && isset($_FILES['imagen'])){

    $nombreArchivo = $_FILES['imagen']['name'];
    $filesize = $_FILES['imagen']['size'];
    $directorioTemp = $_FILES['imagen']['tmp_name'];
    $tipoArchivo = $_FILES['imagen']['type'];
    $arrayArchivo = pathinfo($nombreArchivo);
    $extension = $arrayArchivo['extension'];

    $nombreCompleto = $directorioSubida.$nombreArchivo.".".$extension;
    move_uploaded_file($directorioTemp, $nombreCompleto);
    print "El archivo se ha subido correctamente";

}

?>
```